การตั้งค่าโปรเจกต์ Node.js และการติดตั้ง Middleware พื้นฐาน

Function 3.1 - The Identity Verification Server

1. วัตถุประสงค์

1. เพื่อฝึกการตั้งค่าโปรเจกต์ Node.js และการติดตั้ง Middleware พื้นฐาน
2. เพื่อทำความเข้าใจการรับข้อมูลผ่าน **Query Parameters**
3. เพื่อฝึกการจัดการ **HTTP Status Codes** และการตอบกลับข้อมูลรูปแบบ **JSON**

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. สิ่งที่ต้องเตรียม (Prerequisites)

- ติดตั้ง **Node.js** (LTS Version)
- โปรแกรม **VS Code**
- เครื่องมือทดสอบ API เช่น **Postman** หรือ **Thunder Client** (VS Code Extension)

3. ขั้นตอนการปฏิบัติงาน (Step-by-Step)

ขั้นตอนที่ 1: การเตรียม Folder และ Initialize โปรเจกต์

1. สร้างโฟลเดอร์ชื่อ `lab1-identity-scanner`
2. เปิด Terminal ในโฟลเดอร์นั้นแล้วพิมพ์:

   Bash

   ```bash
   npm init -y
   ```

3. ติดตั้ง Dependencies ที่จำเป็น:

   Bash

   ```bash
   npm install express cors morgan
   ```

4. แก้ไขไฟล์ `package.json` เพิ่มบรรทัดนี้เพื่อใช้ระบบ ESM:

   JSON

   ```json
   "type": "module",
   ```

ขั้นตอนที่ 2: การเขียนโค้ดเริ่มต้น (Code Starter)

สร้างไฟล์ชื่อ `server.js` และใช้เขียน Code

4. การทดสอบผล (Testing)

ทดสอบผ่านเบราว์เซอร์หรือ Postman ดังนี้:

1. **กรณีผ่าน:** เรียกไปที่ `http://localhost:8080/scan?token=admin`
    - ผลลัพธ์ต้องได้ Status `200 OK` และ JSON ตามโจทย์
2. **กรณีไม่ผ่าน:** เรียกไปที่ `http://localhost:8080/scan?token=guest`
    - ผลลัพธ์ต้องได้ Status `401 Unauthorized` และ JSON ตามโจทย์

Checklist Function 3.1

C1. Server สามารถรันบน Port 8080 ได้สำเร็จ

- **วิธีดู:** เมื่อพิมพ์คำสั่ง `node server.js` ใน Terminal จะต้องไม่ขึ้น Error สีแดง และต้องเห็นข้อความ `Identity Scanner Server is running on http://localhost:8080` (ตามที่เขียนไว้ใน `console.log`)
- **จุดที่ต้องระวัง:** หากมี Error ว่า `EADDRINUSE: address already in use :::8080` แสดงว่าเปิดโปรแกรมอื่นค้างไว้ที่พอร์ตนี้ หรือรัน Server ซ้อนกันสองตัว

C2. มีการเรียกใช้ Morgan และเห็น Log ใน Terminal

- **วิธีดู:** หลังจาก Server รันอยู่ ลองเปิดเบราว์เซอร์ไปที่ URL ของเรา แล้วกลับมาดูที่หน้าจอ Terminal
- ผลที่ต้องเห็น: ต้องมีบรรทัดใหม่ปรากฏขึ้นมาเองโดยอัตโนมัติ เช่น:

  GET /scan?token=admin 200 5.123 ms - 45

- **ถ้าไม่ผ่าน:** Terminal จะนิ่งสนิท ไม่มีข้อความใดๆ ขึ้นเมื่อมีการเรียกใช้

C3. ดึงค่าจาก `req.query.token` มาใช้ได้อย่างถูกต้อง

- **วิธีดู:** ตรวจสอบในโค้ดว่าใช้คำสั่ง `req.query.token` (ตัวเล็กทั้งหมด) หรือไม่
- **การทดสอบ:** ลองเปลี่ยนคำใน URL จาก `?token=admin` เป็น `?token=test` แล้วใช้ `console.log(userToken)` ในโค้ดดูว่าค่าเปลี่ยนตาม URL หรือไม่
- **จุดที่มักผิด:** บางคนอาจเผลอใช้ `req.params` หรือสะกดคำว่า `query` ผิด

C4. เมื่อส่ง `token=admin` ระบบตอบกลับเป็น JSON พร้อม Status 200

- **วิธีดู (ใช้ Postman/Thunder Client):** ดูที่แถบ **Status** ต้องแสดงเลข `200 OK` และในส่วนของ **Body** ต้องแสดงข้อมูลในรูปแบบ JSON:

  JSON

  ```
  {
    "status": "authorized",
    "clearance": "high"
  }
  ```

- **จุดที่ต้องระวัง:** บางคนส่งข้อมูลกลับได้ แต่ลืมตั้งค่า Status หรือส่งกลับเป็นข้อความธรรมดา (Plain Text) แทนที่จะเป็น JSON

C5. เมื่อส่งค่าอื่น ระบบตอบกลับเป็น Status 401

- **วิธีดู:** ทดสอบเปลี่ยน URL เป็น `?token=123` หรือ `?token=hello`

- **ผลที่ต้องเห็น:** ใน Postman ตรงแถบ Status ต้องเป็นตัวเลข **401 Unauthorized** (ตัวเลขจะเป็นสีแดง/ส้ม)
- **ความสำคัญ:** ข้อนี้วัดความเข้าใจเรื่อง Logic การใช้ `if-else` และการใช้ฟังก์ชัน `res.status(401)` ซึ่งสำคัญมากในการทำระบบ Security

C6. โค้ดมีการจัดการ import แบบ ES Modules อย่างถูกต้อง

- **วิธีดู:** ตรวจสอบบรรทัดบนสุดของไฟล์ ต้องใช้คำว่า `import ... from ...` แทนที่จะเป็น `const ... = require(...)`
- **จุดเชื่อมโยง:** ข้อนี้จะผ่านได้ก็ต่อเมื่อในไฟล์ `package.json` มีการใส่ `"type": "module"` ไว้แล้วเท่านั้น หากไม่ใส่แต่ใช้ import โปรแกรมจะรันไม่ขึ้นตั้งแต่ข้อแรก

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. หากเราไม่ใส่ `token` ใน URL (เช่น เรียกแค่ `/scan`) ตัวแปร `userToken` ในโค้ดจะมีค่าเป็นอะไร? และจะเกิดผลลัพธ์อย่างไร?
2. Middleware `morgan('dev')` ช่วยให้นักพัฒนาทราบข้อมูลอะไรบ้างเมื่อมีการเรียก API?

การดึงข้อมูลจากตัวเครื่อง Server จริงๆ (System Information) มาสร้างเป็น API Monitoring เบื้องต้น

Function 3.2 - Interactive real-time health monitoring interface

---

1. วัตถุประสงค์

1. เพื่อฝึกการดึงข้อมูลสถานะของระบบ (System Metrics) ผ่าน Node.js Built-in Modules
2. เพื่อทำความเข้าใจการประมวลผลข้อมูลก่อนส่งกลับ (Data Processing)
3. เพื่อฝึกการออกแบบ Response Schema ที่มีความซับซ้อนขึ้น

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. สิ่งที่ต้องเตรียม (Prerequisites)

- โปรเจกต์ที่ติดตั้ง **Express.js** เรียบร้อยแล้ว (สามารถต่อยอดจาก Function 3.1 ได้)
- ความเข้าใจเรื่องหน่วยของข้อมูล (Bytes, Kilobytes, Megabytes)

3. ขั้นตอนการปฏิบัติงาน (Step-by-Step)

ขั้นตอนที่ 1: การวางโครงสร้างโปรเจกต์

1. สร้างไฟล์ใหม่ชื่อ `health-check.js` (หรือใช้ไฟล์เดิมจาก Fuction 3.1 แต่สร้าง Endpoint ใหม่)
2. ศึกษาคำสั่ง Built-in ของ Node.js ที่ต้องใช้:
   - `process.uptime()` : คืนค่าระยะเวลาที่ Server ทำงาน (วินาที)
   - `process.memoryUsage()` : คืนค่า Object ที่บอกการใช้ RAM (หน่วยเป็น Bytes)

ขั้นตอนที่ 2: การเขียนโค้ดเริ่มต้น (Code Starter)

เขียน Code เพื่อให้ API คืนค่าหน่วยเป็น **Megabytes (MB)** และแสดงเวลาที่อ่านง่าย:

4. การทดสอบผล (Testing)

เรียกไปที่ `http://localhost:8080/health`

- **คาดหวัง:** ต้องได้ JSON ที่มีข้อมูลครบถ้วน และค่า `uptime` ต้องเพิ่มขึ้นเรื่อยๆ เมื่อเรา Refresh หน้าจอ (เพราะ Server ทำงานนานขึ้น)

Checklist Function 3.2

C1. Endpoint Correct (GET /health)

- **วิธีดู:** พิมพ์ URL `http://localhost:8080/health` ใน Browser หรือ Postman แล้วต้องมีข้อมูลตอบกลับมา
- **จุดที่ต้องระวัง:** หากเข้า URL อื่นแล้วได้ข้อมูลเดิม หรือเข้า `/health` แล้วได้ `404 Not Found` แสดงว่าตั้งค่า Path ใน `app.get()` ผิดพลาด

C2. Real-time Uptime (ค่าต้องเพิ่มขึ้น)

- **วิธีดู:** ให้กด **Refresh** หน้าจอ Browser รัวๆ (ประมาณ 3-4 ครั้ง)
- **ผลที่ต้องเห็น:** ค่าในฟิลด์ `uptime` ต้องเปลี่ยนไปทุกครั้งที่กด (เพิ่มขึ้นตามวินาทีที่ผ่านไป)
- **ถ้าไม่ผ่าน:** ค่า `uptime` นิ่งสนิทเท่าเดิมตลอดเวลา (แสดงว่านิสิตอาจจะไปเผลอเก็บค่าใส่ตัวแปรไว้ข้างนอกฟังก์ชัน ทำให้มันไม่อัปเดตเมื่อมี Request ใหม่เข้ามา)

C3. Memory Conversion (Bytes to MB)

- **วิธีดู:** ดูที่ฟิลด์ `memory_usage_mb`
- **ผลที่ต้องเห็น:** ตัวเลขควรอยู่ประมาณ **20 - 100** (ขึ้นอยู่กับโปรเจกต์)
- **จุดที่ต้องระวัง:** ถ้าตัวเลขขึ้นมาเป็นหลักล้าน (เช่น `45612348`) แสดงว่านิสิตไม่ได้หารด้วย `1024` สองครั้ง (หรือหาร `1,048,576`) เพื่อแปลงจาก Bytes เป็น Megabytes ครับ

C4. ISO Timestamp (รูปแบบเวลาสากล)

- **วิธีดู:** ดูที่ฟิลด์ `timestamp`
- **ผลที่ต้องเห็น:** ต้องมีตัวอักษร **T** และ **Z** อยู่ในข้อความเวลา เช่น `2026-01-15T10:30:00.000Z`
- **วิธีทำ:** ตรวจดูว่านิสิตใช้คำสั่ง `new Date().toISOString()` หรือไม่ ถ้าใช้ `new Date().toString()` จะได้รูปแบบเวลาท้องถิ่น ซึ่งไม่เป็นมาตรฐานสากลสำหรับ API

C5. JSON Format (ความสมบูรณ์ของข้อมูล)

- **วิธีดู (ใน Postman):** ดูว่ามีการทำ **Syntax Highlighting** (สีของตัวอักษรแยกตามประเภทข้อมูล) หรือไม่
- **จุดที่ต้องระวัง:** โครงสร้างต้องมีเครื่องหมายปีกกา `{}` ครอบคลุมข้อมูลทั้งหมด และ Key ทุกตัวต้องอยู่ภายในเครื่องหมายคำพูด `" "`

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

ความเข้าใจเชิงลึก: ทำไมต้อง Health Check?

ในระบบระดับ Production ตัว **Health Check Endpoint** นี้จะถูกใช้งานโดยเครื่องมือจำพวก **Monitoring Tools** (เช่น UptimeRobot หรือ Prometheus) เพื่อคอยตรวจสอบว่าเซิร์ฟเวอร์ยัง "มีชีวิต" อยู่หรือไม่ หากค่า Memory พุ่งสูงผิดปกติ หรือ Uptime รีเซ็ตเป็น 0 บ่อยๆ (หมายความว่าเซิร์ฟเวอร์ล่มแล้ว Restart ใหม่) ระบบจะแจ้งเตือนวิศวกรทันทีครับ

สรุปภาพรวม : หัวใจสำคัญคือการที่นิสิตสามารถนำ **Internal Data** (ข้อมูลภายในเครื่อง) ออกมานำเสนอผ่าน **HTTP Interface** ได้อย่างถูกต้อง

คำถาม

1. `process.memoryUsage().rss` คือค่าอะไรในระบบปฏิบัติการ และทำไมเราถึงควรเฝ้าระวังค่านี้?
2. หากต้องการให้ `uptime` แสดงผลเป็นรูปแบบ **"HH:MM:SS"** (ชั่วโมง:นาที:วินาที) แทนที่จะเป็นวินาทีเพียวๆ นิสิตจะมีแนวทางในการเขียน Logic อย่างไร?

สร้าง **"Circuit Breaker Middleware"** เพื่อตรวจจับและตัดการเชื่อมต่อชั่วคราวสำหรับ IP ที่เรียกใช้งานเกินกำหนด

Function 3.3 - Circuit Breaker Middleware

1. โจทย์

- Server อาจจะถูกโจมตีด้วยการกระหน่ำ Request (Spamming) จาก IP บางกลุ่ม ทำให้อัตราการตอบกลับช้าลง
- ดังนั้นจะต้องสร้าง **"Circuit Breaker Middleware"** ขึ้นมาเอง **(ห้ามใช้ Library สำเร็จรูป)** เพื่อตรวจจับและตัดการเชื่อมต่อชั่วคราวสำหรับ IP ที่เรียกใช้งานเกินกำหนด

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. เงื่อนไขและข้อกำหนด (Hard Constraints)

1. **Window Time:** 10 วินาที
2. **Limit:** ไม่เกิน 5 ครั้ง ต่อ 1 IP ภายในช่วง Window Time
3. **Response:** หากเกินกำหนด ต้องตอบกลับด้วย Status **429 (Too Many Requests)** พร้อม JSON ระบุเวลาที่เหลือถึงจะเข้าใช้งานได้ใหม่
4. **No External DB:** ให้ใช้ JavaScript Object หรือ Map ใน Memory ของ Node.js เพื่อเก็บสถานะชั่วคราว (State)
5. **Auto Reset:** ข้อมูลการนับต้องถูกล้างทิ้งเมื่อเวลาผ่านไปครบกำหนดเพื่อไม่ให้ RAM เต็ม

3. ขั้นตอน

ขั้นตอนที่ 1: วิเคราะห์โครงสร้างข้อมูล

นิสิตต้องออกแบบ Object เพื่อเก็บข้อมูล เช่น:

JavaScript

```javascript
const requestLog = {
  "127.0.0.1": { count: 1, firstRequestTime: 1705300000000 }
};
```

ขั้นตอนที่ 2: การเขียนโค้ดเริ่มต้น (Code Starter)

เขียน Code ไฟล์ `server.js`

Checklist Function 3.3

C1. IP Identification (การระบุตัวตนผ่าน IP)

- **วิธีดู:** ตรวจสอบในโค้ดว่ามีการใช้ `req.ip` หรือไม่ และใน Object `ipTracker` จะต้องใช้ IP เป็น **Key** หลัก

- **ผลที่ต้องเห็น:** เมื่อเปิด Browser จากเครื่องตัวเอง และลองใช้มือถือ (ต่อ 4G/5G) เข้าลิงก์เดียวกัน (ใช้ IP ต่างกัน) เซิร์ฟเวอร์ต้องนับการเรียกแยกกันอย่างอิสระ
- **ถ้าไม่ผ่าน:** ระบบนับรวมกันหมดทุกคน (แสดงว่านิสิตไม่ได้แยกข้อมูลตาม IP)

C2. Status 429 (การตัดวงจรเมื่อเกินขีดจำกัด)

- **วิธีดู:** ให้นิสิตเปิด Postman หรือ Browser แล้วกด **Refresh รัวๆ** (มากกว่า 5 ครั้งติดต่อกัน)
- **ผลที่ต้องเห็น:** ครั้งที่ 1-5 ต้องขึ้นว่า `"Task completed successfully!"` แต่พอครั้งที่ 6 เป็นต้นไป **ต้อง** ขึ้นข้อความ Error พร้อมกับเลข Status Code **429** ใน Postman
- **จุดที่ต้องระวัง:** หากขึ้นเลข 200 เหมือนเดิมแต่ส่งข้อความว่า Error แบบนี้ถือว่า **ไม่ผ่าน** เพราะไม่ได้ทำตามมาตรฐาน HTTP

C3. Window Reset (การคืนค่าเมื่อครบเวลา)

- **วิธีดู:** หลังจากที่ IP นั้นถูก Block (ขึ้น 429) ให้หยุดส่ง Request แล้วนับในใจไป **11 วินาที**
- **ผลที่ต้องเห็น:** เมื่อกลับมากด Refresh อีกครั้ง ระบบต้องกลับมาใช้งานได้ (Response 200) เหมือนคนเพิ่งเข้ามาครั้งแรก
- **ถ้าไม่ผ่าน:** ระบบ Block ถาวร หรือ Reset ก่อนเวลา 10 วินาทีที่กำหนดไว้

C4. No Memory Leak (การบริหารจัดการ RAM)

- **วิธีดู (ระดับสูง):** ตรวจสอบในโค้ดว่านิสิตมีการใช้คำสั่ง `delete ipTracker[clientIp]` เมื่อผ่านช่วงเวลาไปแล้ว หรือมีการตั้งค่า `setTimeout` เพื่อล้างข้อมูลหรือไม่
- **ความสำคัญ:** หากไม่ล้างข้อมูลเลย Object `ipTracker` จะโตขึ้นเรื่อยๆ จน RAM เต็มในที่สุด หากนิสิตมี Logic การล้างข้อมูลที่ "ไม่ได้ใช้งานแล้ว" ให้ถือว่าผ่านเกณฑ์ข้อนี้ (Bonus)

C5. Accurate Remaining Time (การแจ้งเวลาที่เหลือ)

- **วิธีดู:** ดูที่ JSON ของ Status 429
- **ผลที่ต้องเห็น:** ต้องมีฟิลด์เช่น `"retry_after": 7` (หมายถึงอีก 7 วินาทีจะเข้าได้ใหม่) และเลขนี้ต้องลดลงเรื่อยๆ เมื่อเรายิง Request ซ้ำขณะติด Block
- **วิธีคำนวณ:** นิสิตต้องเอา `(เวลาที่เริ่มยิงครั้งแรก + 10 วินาที) - เวลาปัจจุบัน` แล้วแปลงเป็นวินาที

ตารางเปรียบเทียบพฤติกรรมที่ถูกต้อง

สถานะการเรียก	ครั้งที่ 1-5	ครั้งที่ 6 (ก่อน 10 วินาที)	ครั้งที่ 7 (หลังผ่านไป 10 วินาที)
HTTP Status	`200 OK`	`429 Too Many Requests`	`200 OK`
Response Body	ข้อมูลปกติ	"Too many requests, try again later"	ข้อมูลปกติ (เริ่มนับ 1 ใหม่)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำแนะนำเพิ่มเติม:

เรื่อง **"หน่วยของเวลา"**

- `Date.now()` ให้ค่าเป็น **Milliseconds (ms)**
- โจทย์กำหนด 10 วินาที นิสิตต้องใช้ `10000` ในการคำนวณ
- หากนิสิตเอาวินาทีไปลบกับมิลลิวินาที ระบบจะรวนทันที

คำอธิบาย

- **ข้อ 1-2 (ดูที่ Logic):** นิสิตต้องแสดงให้เห็นว่ามีการใช้ `if (!ipTracker[clientIp])` เพื่อสร้าง Record ใหม่ และ `if (ipTracker[clientIp].count > 5)` เพื่อ Block งาน
- **ข้อ 3 (ดูที่ Time Calc):** ต้องมีการเปรียบเทียบ `now - ipTracker[clientIp].startTime > WINDOW_TIME` เพื่อ Reset ค่า `count` เป็น 1 ใหม่

- **ข้อ 5 (Bonus):** หากนิสิตสามารถส่งค่า `Math.ceil((startTime + WINDOW_TIME - now) / 1000)` กลับไปใน Response ได้ จะแสดงถึงทักษะการคำนวณที่แม่นยำ

คำถาม

1. หากเรา Restart Server ข้อมูลการโดน Block ของ IP ต่างๆ จะยังอยู่หรือไม่? เพราะเหตุใด?
2. การเก็บข้อมูลไว้ใน Object แบบนี้ (`ipTracker`) จะเกิดปัญหาอะไรหากมี IP ที่ไม่ซ้ำกันเรียกเข้ามาหลายล้าน IP ภายในวันเดียว? นิสิตจะมีแนวทางแก้ไขอย่างไร?

การจัดการ **I/O** และ **Memory Efficiency** ด้วย Node.js

Function 3.4 - Secure File Vault Streamer

---

1. โจทย์

- ระบบส่งไฟล์ขนาดใหญ่ (เช่น Log file หรือ Video) ผ่าน API
- โดยมีเงื่อนไขว่า **"ห้ามให้ RAM ของ Server พุ่งสูงเด็ดขาด"** ไม่ว่าไฟล์จะใหญ่แค่ไหน
- ต้องมีการตรวจสอบ **User-Agent** เพื่ออนุญาตให้เฉพาะเครื่องมือที่กำหนดเท่านั้นที่สามารถดึงไฟล์ได้

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. เงื่อนไขและข้อกำหนด (Hard Constraints)

1. **Streaming Only:** ห้ามใช้ `fs.readFile()` หรือ `fs.readFileSync()` (เพราะจะโหลดไฟล์เข้า RAM ทั้งก้อน) ต้องใช้ `fs.createReadStream()` เท่านั้น
2. **Security Header:** ต้องสร้าง Middleware ตรวจสอบ `User-Agent` ใน Request Header หากไม่มีคำว่า `PostmanRuntime` (หรือชื่ออื่นที่อาจารย์กำหนด) ให้ส่ง **403 Forbidden**
3. **Error Handling:** หากระบุชื่อไฟล์ที่ไม่มีจริงในเครื่อง ต้องส่ง **404 Not Found** ในรูปแบบ JSON (ไม่ใช่ปล่อยให้ Error พ่นออกมาในหน้าจอ)
4. **Efficiency:** ระบบต้องสามารถส่งไฟล์ขนาด 100MB+ โดยที่ Node.js Process ใช้ RAM เพิ่มขึ้นไม่เกิน 10-20MB

3. ขั้นตอนการปฏิบัติงาน (Step-by-Step)

ขั้นตอนที่ 1: เตรียมไฟล์ทดสอบ

1. สร้างโฟลเดอร์ชื่อ `vault` ไว้ใน Root ของโปรเจกต์
2. หาไฟล์อะไรก็ได้ที่มีขนาดใหญ่ (เช่น .zip หรือ .mp4) มาวางไว้ในโฟลเดอร์ `vault`

ขั้นตอนที่ 2: การเขียนโค้ดเริ่มต้น (Code Starter)

ไฟล์ `server.js` (นิสิตต้องใช้ความรู้เรื่อง Stream และ Pipe):

Checklist Function 3.4

เน้นไปที่ความเข้าใจเรื่อง **Data Flow** และการจัดการทรัพยากร (Resource Management)

C1. Streaming Protocol (การส่งข้อมูลผ่านท่อ)

- **วิธีดู:** ตรวจสอบในโค้ด **ต้อง** เห็นคำสั่ง `fs.createReadStream()` และตามด้วย `.pipe(res)`

- **จุดที่ต้องระวัง:** หากเห็นคำว่า `fs.readFile` หรือ `fs.readFileSync` แม้โปรแกรมจะส่งไฟล์ได้ แต่ถือว่า **"ตก"** วัตถุประสงค์ของ Function นี้ เพราะนั่นคือการโหลดไฟล์ลง RAM ก่อนส่ง ซึ่งผิดหลักการ Streaming
- **เหตุผล:** การ Pipe คือการส่งข้อมูลทีละส่วน (Chunk) เหมือนการเปิดก๊อกน้ำ ไม่ใช่การตักน้ำใส่ถังจนเต็มแล้วค่อยเท

C2. User-Agent Shield (การตรวจสอบหัวกระดาษ)

- **วิธีดู:** ให้นิสิตลองเปิด Browser (เช่น Chrome/Edge) แล้วพิมพ์ URL ของไฟล์
- **ผลที่ต้องเห็น:** * **ใน Browser:** ต้องขึ้น `{"error": "Forbidden"}` หรือ Status **403**.
  - **ใน Postman:** ต้องสามารถโหลดไฟล์ได้สำเร็จ (เพราะ Postman จะส่ง Header `User-Agent` ที่มีคำว่า `PostmanRuntime` ไปให้โดยอัตโนมัติ)
- **จุดที่มักผิด:** นิสิตสะกดชื่อ Header ผิด (ต้องเป็น `req.get('User-Agent')` หรือ `req.headers['user-agent']`)

C3. Proper 404 Handling (การรับมือเมื่อไม่พบไฟล์)

- **วิธีดู:** ลองพิมพ์ชื่อไฟล์มั่วๆ ใน URL เช่น `/stream/ghost-file.txt`
- **ผลที่ต้องเห็น:** ต้องได้รับ Status **404 Not Found** และ Response ต้องเป็น JSON ที่นิสิตออกแบบไว้
- **จุดที่ต้องระวัง:** หากเซิร์ฟเวอร์ค้าง (Infinite Loading) หรือขึ้น Error สีแดงใน Terminal ของนิสิต แสดงว่านิสิตไม่ได้ใช้ `fs.existsSync` หรือไม่ได้ดัก `stream.on('error')` ไว้

C4. Memory Stability (ความเสถียรของหน่วยความจำ)

- **วิธีดู (วิธีของโปร):** ให้นิสิตเปิด Task Manager (Windows) หรือ Activity Monitor (Mac) แล้วดูที่กระบวนการ `node`
- **การทดสอบ:** ให้นิสิตเริ่มโหลดไฟล์ขนาดใหญ่ (เช่น 500MB+)
- **ผลที่ต้องเห็น:** กราฟการใช้ RAM ของโปรแกรม Node.js ต้อง **"นิ่ง"** หรือเพิ่มขึ้นเพียงเล็กน้อย (หลักไม่กี่ MB) ตลอดช่วงเวลาที่โหลดไฟล์
- **ถ้าไม่ผ่าน:** RAM จะพุ่งสูงขึ้นตามขนาดของไฟล์ที่กำลังโหลด (แสดงว่าแอบใช้ Buffer/readFile)

C5. Content-Type Header (การระบุชนิดไฟล์ - Bonus)

- **วิธีดู:** ดูที่แถบ **Headers** ในผลลัพธ์ของ Postman
- **ผลที่ต้องเห็น:** หากโหลดไฟล์ `.jpg` ค่า `Content-Type` ควรเป็น `image/jpeg` หรือหากโหลด `.pdf` ควรเป็น `application/pdf`
- **วิธีทำ:** นิสิตอาจใช้ Library เช่น `mime-types` หรือเขียน Logic ตรวจนามสกุลไฟล์เอง (แสดงถึงความใส่ใจในรายละเอียดของ API)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

สรุปความแตกต่างที่นิสิตต้องเห็น

คุณสมบัติ	วิธีที่ผิด (readFile)	วิธีที่ถูก (Streaming)
การใช้ RAM	เพิ่มขึ้นตามขนาดไฟล์ (ไฟล์ 1GB = RAM 1GB+)	คงที่ (ใช้ RAM น้อยมากแม้ไฟล์จะใหญ่แค่ไหน)
ความเร็วเริ่มต้น	ต้องรออ่านไฟล์จนจบถึงจะเริ่มส่ง	เริ่มส่งข้อมูลได้ทันทีที่เริ่มอ่าน (First Byte เร็วมาก)
ความเสถียร	เซิร์ฟเวอร์ล่มได้ง่ายหากมีคนโหลดพร้อมกันเยอะ	รองรับการโหลดพร้อมกันได้ดีกว่ามาก

ทำให้ทราบว่าทำไม Node.js ถึงเป็นที่นิยมในการทำระบบ Streaming หรือ Video Platform

---

4. คำอธิบายเกณฑ์การตรวจ (Instructor's Guide)

- **ทำไมต้อง Pipe?:** การใช้ `.pipe(res)` คือการเชื่อมท่อระหว่างไฟล์กับ Network โดยตรง ข้อมูลจะถูกส่งเป็นส่วนๆ (Chunks) ทำให้ RAM ไม่เต็ม

- **การตรวจ RAM:** อาจารย์สามารถเปิด **Task Manager** (Windows) หรือ **Activity Monitor** (Mac) เพื่อดู Process ของ `node` ขณะนิสิตกำลังดาวน์โหลดไฟล์ หาก RAM นิ่งแสดงว่าผ่าน
- **Error Handling:** นิสิตมักลืมใส่ `stream.on('error', ...)` ซึ่งถ้าไฟล์มีปัญหาระหว่างส่ง จะทำให้ Server ล่มได้

---

5. คำถาม

1. การใช้ `fs.readFile()` กับไฟล์ขนาด 2GB บน Server ที่มี RAM 1GB จะเกิดผลอย่างไร?
2. `stream.pipe(res)` มีข้อดีอย่างไรเมื่อเทียบกับการรออ่านไฟล์ให้เสร็จก่อนแล้วค่อยส่ง (Buffer)?
3. เราสามารถใช้ Middleware ในข้อนี้เพื่อจำกัดสิทธิ์ให้โหลดไฟล์ได้เฉพาะสมาชิก (Member) ได้อย่างไร

Asynchronous Programming, External API Integration และ Data Transformation

---

Function 3.5 - Simulation of dynamic API and reverse proxy

---

1. โจทย์

- สร้าง **API Gateway** เพื่อไปดึงข้อมูลจาก Server ภายนอก (Public API)
- แต่ปัญหาคือข้อมูลต้นทางนั้นมีโครงสร้างที่ "ซับซ้อนเกินความจำเป็น" และมี Key เป็นภาษาอังกฤษที่เข้าใจยาก
- ต้องสร้าง Server มาคั่นกลางเพื่อดึงข้อมูลนั้นมา **"กรอง"** และ **"แปลงร่าง"** (Transform) ให้เป็น Format ใหม่

หมายเหตุ Final Project จะต้องเป็นแอปที่มีฟีเจอร์เทียบเท่าฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist ด้านล่างนี้ได้ผ่านทุกข้อ

2. เงื่อนไขและข้อกำหนด (Hard Constraints)

1. **External Fetch:** ต้องใช้ `fetch` (Built-in) หรือ Library `axios` ในการดึงข้อมูลจาก Public API (แนะนำ: PokeAPI หรือ JSONPlaceholder)
2. **Data Transformation:** ห้ามส่ง JSON ต้นฉบับกลับไปเด็ดขาด ต้องสร้าง Object ใหม่ที่มีโครงสร้างตามที่กำหนดเอง (เช่น เปลี่ยนชื่อ Field จากอังกฤษเป็นไทย หรือคำนวณค่าใหม่)
3. **Custom Header Injection:** ต้องมีการเพิ่ม Header พิเศษชื่อ `X-Powered-By` และ `X-Student-ID` ลงใน Response
4. **Error Handling:** หาก API ต้นทางล่ม หรือ URL ผิด ต้องจัดการด้วย `try-catch` และส่ง Status **502 Bad Gateway** พร้อมข้อความอธิบายที่สุภาพ

3. ขั้นตอนการปฏิบัติงาน (Step-by-Step)

ขั้นตอนที่ 1: เลือกเป้าหมาย (Public API)

เลือก API ที่สนใจ เช่น `https://jsonplaceholder.typicode.com/users/1` ซึ่งจะได้ข้อมูล User ที่มีทั้ง Address, Company ซ้อนกันหลายชั้น

ขั้นตอนที่ 2: การเขียนโค้ดเริ่มต้น (Code Starter)

ไฟล์ `server.js` (เน้นการใช้ `async/await`):

Checklist Function 3.5

C1. Asynchronous Logic (การรอข้อมูลข้าม Server)

- **วิธีดู:** ตรวจสอบคำว่า `async` หน้า Handler และ `await` หน้าคำสั่ง `fetch`
- **เหตุผล:** การดึงข้อมูลจาก Server อื่นต้องใช้เวลา หากไม่ใช้ `await` ตัวแปรจะกลายเป็น `Promise <pending>` และส่งข้อมูลว่างเปล่ากลับไป

C2. Transformation Success (การแปลงร่างข้อมูล)

- **วิธีดู:** เปิดหน้าเว็บ API ต้นทางเทียบกับ API ของนิสิต
- **ผลที่ต้องเห็น:** โครงสร้างต้อง "แบนลง" (Flatten) หรือ "อ่านง่ายขึ้น" เช่น ต้นฉบับมี `address: { street: "...", suite: "..." }` แต่นิสิตส่งกลับมาเป็น `location: "street, suite"` บรรทัดเดียว

C3. Header Injection (การประทับตรา)

- **วิธีดู:** ใน Postman ดูที่แท็บ **Headers** (ข้างๆ แท็บ Body)
- **ผลที่ต้องเห็น:** ต้องมี Key ชื่อ `X-Student-ID` ปรากฏอยู่พร้อมค่ารหัสนิสิต นี่คือการยืนยันว่า Request นี้ผ่าน Gateway ของนิสิตมาจริงๆ ไม่ใช่การเรียกตรงไปที่ต้นทาง

C4. Robust Error Handling (ความทนทาน)

- **วิธีดู:** ให้นิสิตแกล้งพิมพ์ URL ของ API ต้นทางให้ผิด (เช่น ลบตัวอักษรออกตัวหนึ่ง)
- **ผลที่ต้องเห็น:** Server ของนิสิตต้องไม่หยุดทำงาน (ไม่ขึ้นตัวหนังสือสีแดงใน Console) และผู้ใช้งานต้องเห็น JSON สวยๆ ที่บอกว่า `Bad Gateway` พร้อม Status **502**

C5. Dynamic Path (ความยืดหยุ่น)

- **วิธีดู:** ทดสอบเปลี่ยน ID ท้าย URL
- **ผลที่ต้องเห็น:** ข้อมูลที่ตอบกลับมาต้องเปลี่ยนไปตาม ID นั้นๆ (แสดงว่านิสิตใช้ `req.params` ไปต่อยอดใน `fetch` ได้ถูกต้อง)

<u>หมายเหตุ</u> Final Project จะต้องเป็นแอปที่มีฟีเจอร์<u>เทียบเท่า</u>ฟังก์ชั่นเหล่านี้ แล้วตรวจตาม Checklist นี้ได้ผ่านทุกข้อ

คำถาม

1. ทำไมเราถึงต้องมี API Gateway มาคั่นกลาง แทนที่จะให้ Frontend เรียกหา Public API โดยตรง? (ใบ้: เรื่องความปลอดภัยและ CORS)
2. หาก API ต้นทางตอบกลับช้ามาก (เช่น 30 วินาที) จะส่งผลอย่างไรต่อ Node.js Server ของเรา?
3. นิสิตคิดว่าการทำ Data Transformation มีประโยชน์อย่างไรในแง่ของการลด Bandwidth ของ Mobile App?